

EXAM 8

Software quality is a way to measure overall how much software conforms to the requirements a meets the users need.

To guarantee some form of quality there is the SQA, which is a set of procedures and policies so that components, software, and procedures by themselves can certifiably be considered 'of quality'.

For this reason we have different standards created.

ISO 9000 has been created mostly for quality in organizations procedures such that it's products satisfy quality.

(ISO 9126 → ISO 25010)

Software quality ~~is~~ doesn't touch just code, but deals with:

- Requirements
- Code
- Documentations
- Associated Data
- Procedures

The goal of software quality is to lessen the frequency of mistakes:

- error
- faults
- failures.

Errors can happen for multiple reasons, some of them are:

- Errors in the requirements
- Errors in customer-developer relationship
- Errors in code
- Errors in documentation / manual
- Errors in wrong code reuse
- Reducing test phases ← happens often!!!
- Design errors

Some of the most common error in code can be associated to:

- not correctly checking data types
- Data declaration errors
- data conversion errors → Spaceship incident
- comparison errors
- control flow / algorithmic errors.

Achieving software quality is expensive, so there is tradeoff between cost of conformity and cost of insufficient quality.

Standards like the ISO25010 have been created to tackle software quality and give some directives on how to test have quality.

ISO 25010 makes two different sections

Quality of use

- Effectiveness
- Efficiency
- Satisfaction
- Context Coverage
- Freedom from risks.

Product quality

- reliability
- portability
- security
- maintainability
- efficiency
- usability
- compatibility
- suitability



Exam 12.09.2022

Q3:

way to objectively determine how much our code needs to do when it's still at the requirements parts of the development process.
It's based on 5 main points.

- ILF Internal logical file, that has at its inside the data and logical part of it
- EIF External interface file; which has reference to a set of dates and interfaces. EIF $\approx$ ILF
- EI External input
- EO External output
- EQ External query.

the use of this is to transfer varying levels of this point, with every FP assigned a difficulty value 1...5, to actual LOC (lines of code).

INPUT (LOC)

↳ COCOMO & COCOMO II is a way to have levels of accuracy. They both aim at estimating LOC using estimation of both time and number of people needed (person-months).

COCOMO and its first revision, uses a waterfall model and a monolithic design language. It's usually pretty optimistic and with 15 cost factors aims at creating a perfect estimation.
(organic, semidetached, embedded)
with cost factors (15)

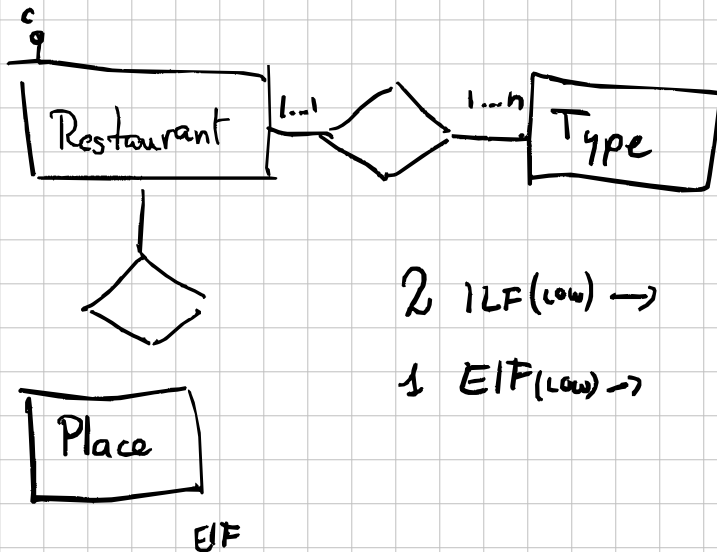
↳ Time has proven that estimation is usually 15% to 50% different than reality!

COCOMO II is the answer to this, providing a more complete approach by having 2 different estimation models with varying levels of accuracy. (1 with 7 cost factors, other with 17) with also a way of calibrating results to more closely follow plan.

↳ Better version that includes the agile paradigm and has removal of software in mind.

It's non linear and aims to provide a range, more than an actual point estimation.

Function Points



E1

1.	2	FTR	10	DET	→	medium	6 FP
2.	1	FTR	1	DET	→		3 FP
3.	1	FTR	1	DET			3 FP

NON VO VOGELAAA

COCOMO

we have 2 different estimated formulas

T time needed
M: Person } This helps to estimate manpower,

we have three different models, based on what kind of project we have

- ORGANIC

- SEMI DETACHED

- EMBEDDED

an 3 different level of depth regarding estimates
base

medium (15 cost points, rated 1...5)

hard every cost has correctional coefficient

it's not very useful nowadays, since its based on waterfall model alone for time partitioning

Requirements - Design - Development - Integration & Test

COCOMO II

Expands on this, developing more on code reuse, team cohesion and more agile models.

We have here too 2 models for different precision.

Early design
- 7 cost factors

Post-arch. model
17 cost factors

we have 5 SCALING DRIVERS as exponent
adjustment

PRECEDENTNESS

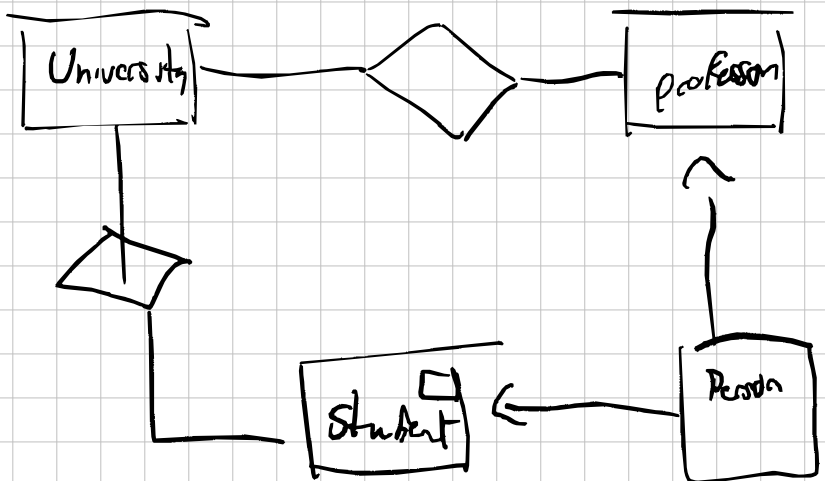
FLEXIBILITY

RESILIENCE

TEAM COHESION

PROJECT MATURITY

SCRUM : Infostud



As a professor, i want to give grades on my students

As a sysadmin, i want to manage courses and professors on courses

as a professor i want to access classes schedule to change, add, modify

as a student i want to manage my bookings

as a student i want to have way to show how i am doing

as a sysadmin i want to be able to make students pay their taxes on the site itself.

REST interface

GET PUT	/classes		<u>class /id/</u>	GET - PUT - POST - DELETE -
	/users		user /id/ ↳ bookings ↳ grades	
	/professors		professor /id/ /bookings	